

# Code Optimization con .NET Profiler

Giulio Sciarappa

# Sponsor


CONSORTIVM SERVICE S.r.l.


UNIPORDENONE
moltiplica il valore della formazione

# Il tempo perso




*"L'app è lenta." - Tre parole. Tre giorni di lavoro.*

# Il messaggio che tutti conosciamo

Teams – Venerdì ore 17:59

**Cliente:** "L'app è lenta."

*Tu: Quale app? Lenta quanto? Lenta per chi? Lenta quando?*

E così inizia la caccia al fantasma...

# Il problema delle performance



## Si nascondono

- ✓ Non si vedono nei test
- ✓ Non si vedono in staging
- ✓ Non si vedono in locale
- ✓ Appaiono solo in produzione
- ✓ *Quando nessuno guarda ☺*

# La cassetta degli attrezzi tradizionale



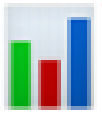
## LOGGING

```
Console.WriteLine
_logger.LogInfo
Debug.Print
```



## STOPWATCH

```
var sw = new
Stopwatch();
sw.Start();
```



## APM BASE

```
Response time
Error rate
Throughput
```

# La cassetta degli attrezzi tradizionale




- 😰 Devi SAPERE dove guardare
- 😰 Modifica codice → deploy → aspetta
- 😰 Troppi log = rumore

- 😰 Invasivo: modifichi il codice
- 😰 Non cattura GC, thread, I/O
- 😰 Misuri UN path

- 😰 Ti dice COSA, non PERCHÉ
- 😰 "Request: 3s" → ok ma DOVE?
- 😰 Black box

# Il Profiler in 30 secondi

## Application Insights Profiler

Uno strumento che registra cosa sta facendo il vostro codice,
metodo per metodo, linea per linea, mentre gira in produzione.

🔍 Cattura stack trace delle request lente

📊 Visualizza con flame graph interattivi

⚡ Overhead minimo (~2%)

🚀 Zero modifica al codice

# Come funziona: Sampling




**PROFILER LOGIC**

Request > soglia?

✅ SI → CATTURA TRACE

❌ NO → IGNORA

**📸 CATTURA STACK TRACE**

Solo per request > soglia

(default: 1 secondo)

# Attivare il profiler: 3 step

1

## Application Insights configurato

```
builder.Services.AddApplicationInsightsTelemetry();
builder.Services.AddServiceProfiler();
```

2

## Abilita nel portale Azure (quando previsto)

```
App Insights → Performance → Profiler → ON
```

3

## (Opzionale) Trigger personalizzato

```
Default: request > 1 secondo
```

# Matrice di compatibilità e scenari

## Azure App Service

| Scenario | Supportato | Metodo | Note |
|---|---|---|---|
| **Windows + ASP.NET** | ✓ | Codeless (portale) | Basic tier o superiore. Preinstallato nel runtime. |
| **Windows + ASP.NET Core** | ✓ | Codeless (portale) | Basic tier o superiore. Segue .NET Core support policy. |
| **Linux + ASP.NET Core** | ✓ | SDK NuGet | Richiede pacchetto Microsoft.ApplicationInsights.Profiler.AspNetCore |
| **Free / Shared tier** | ✗ | N/A | Richiede minimo Basic tier per Always On |
| **Custom Containers** | ✗ | N/A | Solo deploy "as Code" supportato |

# Matrice di compatibilità e scenari

## Azure Functions

| Scenario | Supportato | Metodo | Note |
|---|---|---|---|
| **Premium Plan (EP1, EP2, EP3)** | ✓ | SDK NuGet | Funziona come App Service dedicato |
| **Dedicated Plan (App Service)** | ✓ | SDK NuGet | Basic tier o superiore richiesto |
| **Consumption Plan** | ✗ | N/A | NON supportato - nessuna VM persistente |

# Matrice di compatibilità e scenari

## Azure Compute (VM / VMSS)

| Scenario | Supportato | Metodo | Note |
|---|---|---|---|
| **Azure VM (Windows)** | ✓ | Azure Diagnostics Extension | Configurare WADCfg sink nel template ARM |
| **Azure VMSS (Windows)** | ✓ | Azure Diagnostics Extension | Stessa configurazione delle VM singole |
| **Azure VM (Linux)** | ✓ | SDK NuGet | Richiede integrazione manuale nell'applicazione |

# Matrice di compatibilità e scenari

## Containers e Kubernetes

| Scenario | Supportato | Metodo | Note |
|---|---|---|---|
| **Docker (Linux)** | ✓ | SDK NuGet | Aggiungere pacchetto nel Dockerfile |
| **AKS (Azure Kubernetes)** | ✓ | SDK NuGet | Integrazione tramite codice applicativo |
| **Azure Container Apps** | ✓ | SDK NuGet | Come altri container Linux |

# Matrice di compatibilità e scenari

## Runtime e Framework .NET

| Runtime | Supportato | Versione Profiler | Note |
|---|---|---|---|
| **.NET 8** | ✓ | 2.7.0+ | Raccomandato - LTS |
| **.NET 7** | ✓ | 2.5.0+ | STS - fuori supporto Nov 2024 |
| **.NET 6** | ✓ | 2.4.0+ | LTS - supporto fino Nov 2024 |
| **.NET 5** | ✗ | Deprecato | Fuori supporto - aggiornare a .NET 6+ |
| **.NET Core 3.1** | ✗ | Deprecato | Fuori supporto - aggiornare a .NET 6+ |
| **.NET Framework 4.x** | ✗ | N/A | NON supportato - solo .NET Core/5+ |

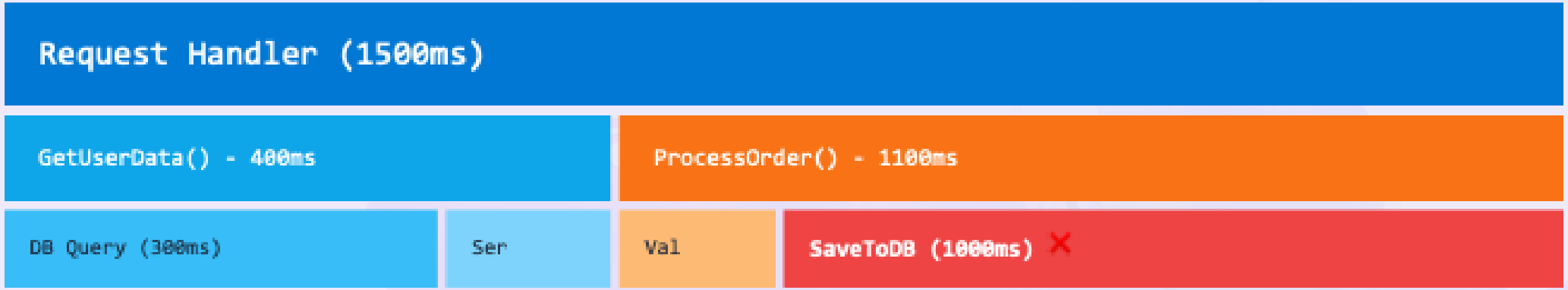
# Requisiti e configurazioni

| Requisito | Dettagli |
|---|---|
| **Always On** | Deve essere abilitato in App Service (richiede Basic tier+) |
| **Connection String** | APPLICATIONINSIGHTS_CONNECTION_STRING configurata correttamente |
| **Tier minimo** | Basic tier o superiore per App Service / Premium per Functions |
| **Traffico in ingresso** | Il Profiler cattura trace solo quando ci sono request attive |
| **Firewall / Network** | Deve permettere connessioni agli endpoint di ingestion di Application Insights |
| **ETW Sessions (Windows)** | Limite di 1 sessione di profiling per VM - include Diagnose, Kudu e Profiler |
| **Overhead** | 5-15% CPU durante la raccolta (2 minuti ogni ora circa) |
| **Retention dati** | I trace sono eliminati dopo 15 giorni automaticamente |

# Demo



# Leggere un flame graph




## REGOLE D'ORO

### LARGHEZZA = Tempo speso

Più largo = più lento = guarda qui

### ALTEZZA = Profondità stack

Il metodo sopra ha chiamato quello sotto

### OBIETTIVO

Trovare i blocchi larghi in basso ("mangiano" tempo)

# Los tres caballeros (cit)



### 1 IL MURO

Un singolo metodo domina

*Query lenta, I/O bloccante*

### 2 LA SCALA

Stesso metodo ripetuto

*N+1 query, loop inefficiente*

### 3 IL VUOTO

attesa

Spazi vuoti tra blocchi

*Servizio esterno, lock*

# E finalmente… Code Optimizer

**FLAME GRAPH**

`SaveToDB (1000ms)`

❓ Perché è lento?



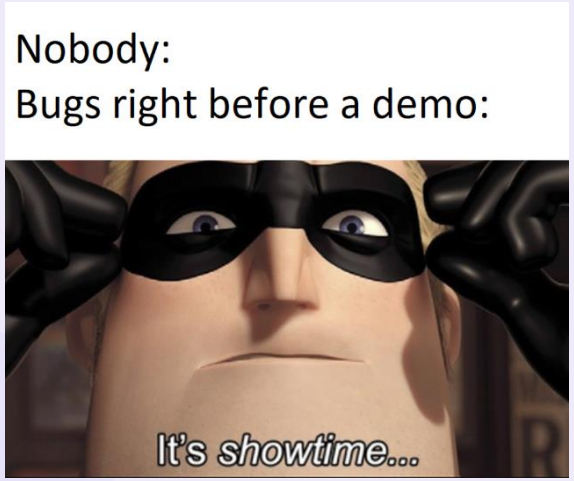
**🤖 CODE OPTIMIZER**

"Questo metodo è lento perché
stai facendo una INSERT
per ogni record."

💡 **Usa SqlBulkCopy**

- 🤖 Analizza trace + codice sorgente
- 📝 Spiega PERCHÉ è lento
- 💡 Suggerisce COME migliorarlo

# Demo



# I soliti sospetti 

### 🥇 N+1 QUERY

Sintomo: Pattern "Scala", stesso metodo DB ripetuto — **Fix: Include(), Join, batch loading**

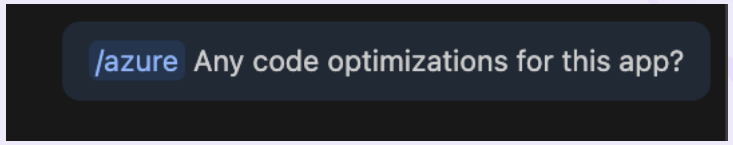
### 🥈 SYNC OVER ASYNC

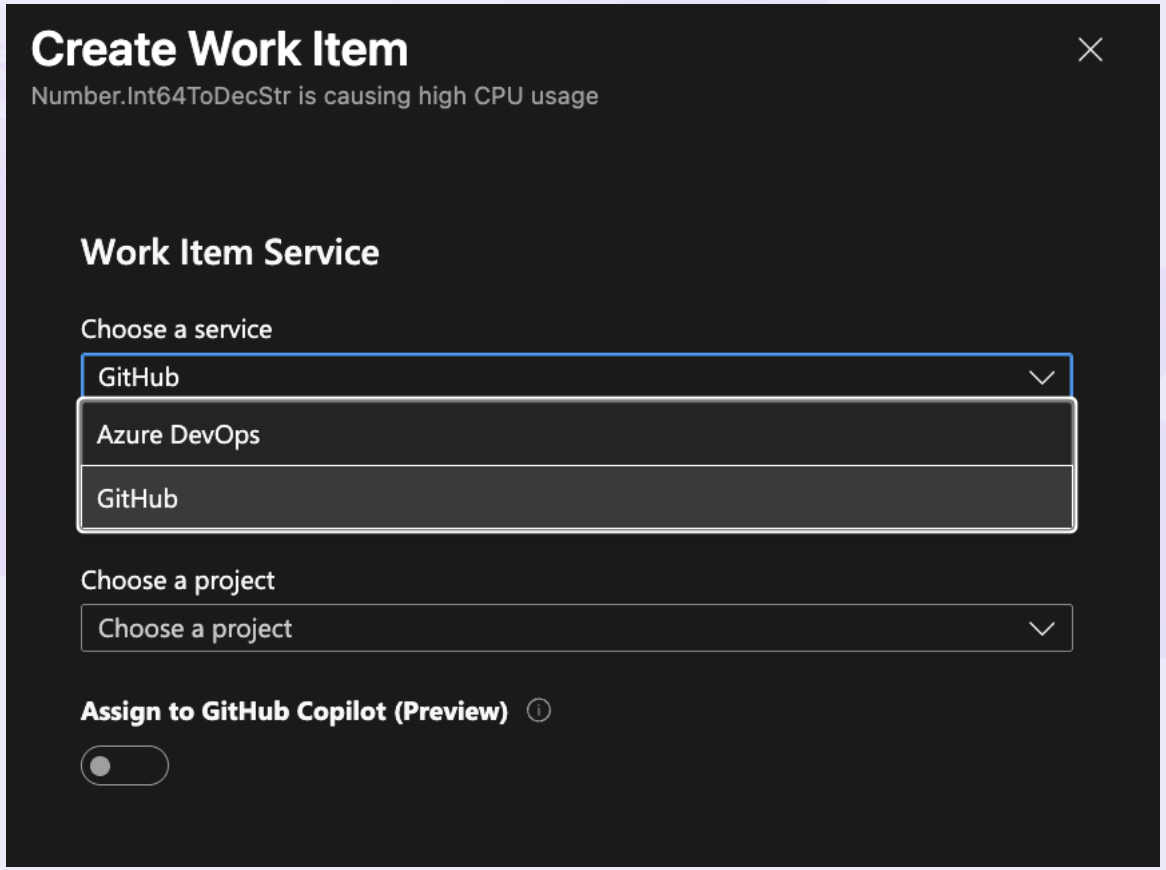
Sintomo: "Muro" con .Result o .Wait() nello stack — **Fix: async/await fino in fondo**

### 🥉 STRING CONCATENATION

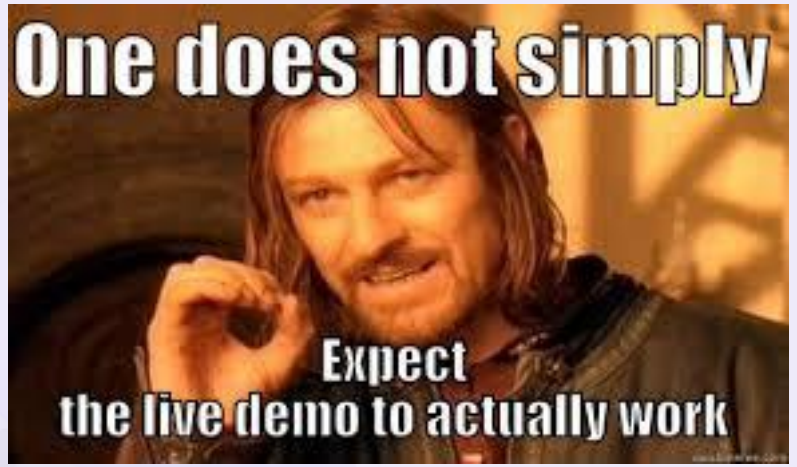
Sintomo: Tempo in System.String, allocazioni massive — **Fix: StringBuilder o string.Join**

# Fix: Github Copilot! (preview)



# Demo



# Bonus: OpenTelemetry Profiler

| Caratteristica | App Insights SDK | OpenTelemetry Profiler |
|---|---|---|
| **Standard** | Proprietario Microsoft | CNCF OpenTelemetry ✓ |
| **Pacchetto NuGet** | Microsoft.ApplicationInsights.Profiler.AspNetCore | Azure.Monitor.OpenTelemetry.Profiler |
| **Stato** | GA (Stabile) | Preview (Beta 7) |
| **Runtime minimo** | .NET 6+ | .NET 8+ |
| **Vendor Lock-in** | Sì (Azure only) | No (standard aperto) |
| **Integrazione OTel** | Limitata | Nativa ✓ |

```
// Program.cs
builder.Services
  .AddOpenTelemetry()
  .UseAzureMonitor()
  .AddAzureMonitorProfiler();
```

# Il ciclo completo




**DETECT → ANALYZE → FIX**

# Risorse

## DOCUMENTAZIONE

| | |
|---|---|
| Application Insights Profiler | aka.ms/aiprofiler |
| Code Optimizations | aka.ms/codeoptimizations |
| Troubleshooting | aka.ms/profiler-troubleshoot |
| Presentazione | tinyurl.com/PresentazioniGiulio |

# Grazie mille!



# Domande?

